RSLAB

A Pure-Rust Sparse Direct Solver with a Learned, Memory-Guaranteed Auto-Tuner

Milan Rother

July 1, 2026

Abstract

RSLAB is an embeddable, pure-Rust sparse *direct* solver: a supernodal complex-symmetric $\mathbf{LDL}^\top$ factorization with Bunch–Kaufman pivoting and an unsymmetric multifrontal LU, exposed through a small Python API. It is type-agnostic over the scalar field (`f64`, `f32`, `Complex<f64>`, `Complex<f32>`) and carries no BLAS/LAPACK, METIS, or Fortran dependency — every kernel, ordering, and the tuner itself are Rust. Two design commitments distinguish it. First, the symbolic analysis produces a *per-path a-priori peak-memory estimate* — a separate deterministic bound for the left-looking and the multifrontal schedule — that is a guaranteed upper bound on the measured peak (geomean ≈1.5×, never under-predicting across the corpus). Second, on top of that estimate sits a *learned* auto-tuner: a small multilayer perceptron predicts factor time and peak memory from the matrix's structural fingerprint and a candidate knob vector, and picks the configuration minimizing a weighted Pareto objective — but every pick is filtered through a deterministic guard stack that makes it provably never use more memory than the untuned default, and falls back to that default outside the training distribution. The learned component is thus *speculative*, like a CPU branch predictor, while a deterministic safety net keeps misspeculation free of cost. Over a 36-matrix SuiteSparse corpus the auto-tuned default is 1.38× faster than the untuned solver at 0 of 89 memory regressions, beats faer by 7.0× (time) and 2.3× (memory) and SuperLU by 8.9×, and lands within a single-digit factor of Intel MKL PARDISO. We describe the algorithms, the estimator, the tuner and its guard stack, and the deterministic resource governor that turns the a-priori estimates into full, reproducible RAM/CPU utilization for batched solver-in-the-loop workloads.

1 Scope

RSLAB factorizes and solves sparse linear systems $Ax = b$ where A is either complex-symmetric (routed to $\mathbf{LDL}^\top$) or general unsymmetric (routed to LU). The target regime is the complex-symmetric systems of frequency-domain electromagnetic and FEM analysis — in particular *curl–curl* (Nédélec edge-element) Maxwell problems with lossy media or PML, which are complex-symmetric rather than Hermitian. It is written in stable Rust with no C/Fortran dependency and no BLAS/LAPACK/METIS linkage; the SIMD dense kernels, the fill-reducing orderings (AMD, AMF, nested dissection), the factorization, and the auto-tuner are all Rust, and the solver is exposed to Python through a PyO3 layer. This report states the architecture, the numerical kernels, the a-priori resource model, the learned auto-tuner and its safety guards, and reports the solver against faer, MKL PARDISO, and SuperLU over the SuiteSparse collection.

2 Architecture

The solver is a Cargo workspace (Table 1). The pipeline is the classical three-phase sparse-direct flow — *analyze* (order, build the elimination tree and supernodes, estimate memory), *factor* (numeric $\mathbf{LDL}^\top$/LU), *solve* (triangular solves) — with the analyze phase additionally producing the structural feature vector and memory estimate that drive the auto-tuner and the resource governor.

Crate / module	Responsibility
`rslab-ordering-core`	Shared quotient-graph elimination engine (workspace, garbage collection, supervariables)
`rslab-amd`	Approximate minimum degree (Amestoy–Davis–Duff)
`rslab-amf`	Approximate minimum fill (HAMF4, Amestoy 1999)
`rslab-metis`	Multilevel nested dissection (iterative driver, König separators)
`symbolic`	Elimination tree, supernode amalgamation, column counts, ordering dispatch
`numeric`	Supernodal $\mathbf{LDL}^\top$ (Bunch–Kaufman), multifrontal LU, panel-freeing, scoped pools
`diagnostics`	A-priori per-path memory + flop estimates
`analysis`	Structural feature extraction, thread-count predictor
`auto_tune`	Embedded MLP performance model + guard stack (pure-Rust inference)
`tuning`	Hardware calibration + budget-aware resource governor
`python`	PyO3 bindings

Table 1: Workspace layout. The `rslab-*` ordering crates and the `numeric` core are pure Rust with no external numeric library; `python` is a PyO3 crate depending on them.

3 Symbolic analysis and ordering

3.1 Fill-reducing orderings

The ordering choice dominates fill and hence both factor time and memory. RSLAB ships three families behind an `OrderingMethod` enum and an adaptive dispatcher.

Approximate minimum degree (AMD). A quotient-graph elimination with approximate external degree, aggressive element absorption, and supervariable detection (Amestoy–Davis–Duff [1]). The workspace maintains the element/variable lists in a single sliding index array with in-place garbage collection, degree-indexed LIFO bucket lists, hash-bucket supervariable detection, and mass elimination. Dense columns of degree exceeding $\max(16, \lfloor \alpha\sqrt{n} \rfloor)$ with $\alpha = 10$ are deferred to the tail. It is the default for very large, very sparse patterns ($n > 100\,000$, average degree < 5), where nested dissection's separator overhead does not pay off.

Approximate minimum fill (AMF). The HAMF4 variant [2, 3] scores each supervariable by approximate *fill* rather than degree,

$$\mathrm{RMF}(i) = \frac{\deg(i)\left(\deg(i) - 1 + 2\deg_{me}\right) - \mathrm{WF}(i)}{nv(i) + 1}, \tag{1}$$

where $\mathrm{WF}(i)$ is the accumulated working fill avoided by absorptions. Working fill is accumulated in 64-bit arithmetic (the product is $\mathcal{O}(n^2)$ and overflows 32-bit for $n \gtrsim 46\,000$). AMF is the default for $n \le 10\,000$, where it stays within $1.10\times$ of MUMPS' HAMF4 fill on a 183 277-matrix oracle suite.

Nested dissection (ND). A multilevel recursive bisection [4, 5] with an *iterative*, non-recursive driver: an explicit work stack of (subgraph, vertex map, offset) items, so deep dissection trees cannot overflow the call stack. Each level coarsens the graph, computes an initial bisection refined by Fiduccia–Mattheyses [6], uncoarsens with projection + refinement, and extracts a minimum vertex-cover separator by König's theorem on the bipartite crossing-edge graph. Two guards keep it robust: subgraphs below `nd_to_amd_switch` $= 200$ vertices are ordered directly by AMD, and a degenerate split (one side $\ge 90\%$ of the vertices) falls back to an AMD leaf rather than recursing on a graph without a good separator. ND is the default for $n > 10\,000$.

Dispatch. `OrderingMethod::Auto` routes by cheap structural predicates: very-large-and-sparse $\rightarrow$ AMD; an arrow/bordered pattern (few high-degree "border" columns carrying a large share of the nonzeros, detected by $\geq 20\%$ of nnz concentrated in $< 5\%$ of columns above a degree floor of $\max(64, 8\,\overline{\deg})$) $\rightarrow$ AMF, which defers the dense rows to the trailing block where ND would smear them across separators at 7–9$\times$ the fill; otherwise the size rule above. `OrderingMethod::AutoRace` instead *measures*: it runs the full symbolic factorization for each concrete candidate and keeps the one with the smallest estimated factor nnz, at $\sim$4$\times$ the single-pass analysis cost, paid once per problem.

3.2 Elimination tree, supernodes, amalgamation

From the ordered pattern RSLAB builds the elimination tree and detects fundamental supernodes (maximal runs of columns with identical row structure). Small supernodes are *amalgamated* to widen the dense fronts that feed the BLAS-3 kernels: nodes with fewer than `nemin` $= 16$ columns (SSIDS default [7]) are merged with their parent under a column-renumbering scheme that re-postorders the tree so desired merges become structurally adjacent. Beyond that, *relaxed* amalgamation (`RelaxAmalgamation`, applied for $n \geq 1024$) merges an adjacent child into its parent even when not fill-justified, as long as the merged supernode stays within a width and extra-row budget — trading a little explicit-zero fill for wider, higher-rank Schur updates [8]. These knobs (`nemin`, the relaxation widths, the ordering) are exactly the analyze-time levers the auto-tuner searches over.

4 Numeric factorization

4.1 Equilibration

Before factoring, A is symmetrically equilibrated as $\hat{A} = DAD$ with a real diagonal $D = \mathrm{diag}(s)$, $s_i = 1/\sqrt{\max_j |A_{ij}|}$, computed in one pass over the lower triangle; an all-zero row is left unscaled. The real, symmetric, positive diagonal preserves complex symmetry and the pivot signs, tolerates zero diagonals (ubiquitous in saddle-point systems), and improves the conditioning that the bounded pivoting sees. The solve then unwinds $x = D\,(\hat{A}^{-1}(Db))$.

4.2 Complex-symmetric LDL$^\top$ with Bunch–Kaufman pivoting

The default path is a supernodal *left-looking* **LDL**$^\top$. Each supernode's dense panel is assembled from A, updated by every previously factored descendant (`cmod`: pull the descendant's columns, apply its block-diagonal D, subtract the rank update), then factored in place (`cdiv`: a blocked Bunch–Kaufman partial factorization). Pivoting uses the classical Bunch–Kaufman [9] $1 \times 1/2 \times 2$ scheme with threshold $\alpha = (1 + \sqrt{17})/8 \approx 0.6404$, bounded to each panel's fully-summed rows so the factorization stays within the supernode. For complex-symmetric A the control flow is that of the real `sytf2` with magnitudes $|z|$ and *no conjugation* (the analogue of LAPACK `zsytf2`); a 2×2 block is rejected as singular when $|d_{11}d_{22} - d_{21}^2| < \varepsilon \max(|d_{11}|, |d_{21}|, |d_{22}|)^2$ with $\varepsilon = 1 \times 10^{-14}$, bounding element growth. The panel blocking width is `panel_nb` $= 64$; each panel's trailing Schur update is deferred into a single SIMD GEMM.

4.3 Panel-freeing: the low-transient schedule

The memory advantage of the left-looking path comes from *freeing each dense panel the instant its last consumer is done with it.* Each supernode carries a refcount of the ancestors that still need it; when a descendant's update decrements that count to zero, the panel is compacted into its final CSC factor fragment and the dense buffer is deallocated (coordinated by acquire/release atomics for the tree-parallel factor). The resident data is therefore the compact factor plus only the *live* panels, not all panels at once — the profile that gives RSLAB its low peak memory (Sec. 5).

4.4 Multifrontal path

The alternative `FactorMethod::Multifrontal` [10, 11] assembles a dense frontal matrix per supernode from the eliminated columns and its children's contribution blocks (CBs), then extends the Schur complement to its parent. A work-stealing tree recursion factors children in parallel and frees each child's CB the moment it is assembled into the parent, keeping only the active contribution frontier live. Where the CB stack is large ($\sum_{\text{children}} cb \geq 4 \times 10^6$ scalars) children are reordered by Liu's (peak $-$ cb) rule [12] to shrink the transient spike while preserving leaf parallelism. The multifrontal front is denser and more BLAS-3-rich but carries a higher transient peak than left-looking; the tuner chooses between them per matrix.

4.5 Unsymmetric LU

General matrices reuse the symmetric multifrontal machinery on the symmetrized pattern $A \cup A^\top$, with UMFPACK-style [13] threshold partial pivoting: within each front's fully-summed block a diagonal candidate is accepted when $|a_{jj}| \geq \tau\,|a_{\text{colmax}}|$ with $\tau = 0.1$, else the column maximum is swapped up. The panel width is narrower (`NB` $= 32$) because the SIMD GEMM is fast while the serial `getf2` is not. L is stored CSC (unit lower), U CSR, with a two-sided equilibration.

4.6 Static pivoting for a never-fail preconditioner

The `ZeroPivotAction` policy governs near-singular pivots: `Fail` (the exact default) returns `NumericallyRankDeficient`; `PerturbToEps{abs_floor}` lifts any pivot of magnitude below the floor to that floor while preserving its phase, $d \mapsto d \cdot (\text{abs\_floor}/|d|)$, yielding $\mathbf{LDL}^\top = A + \Delta$ with bounded Δ — the never-fail factor a preconditioner needs. (The iterative-refinement / Krylov path that consumes this factor is outside the scope of this report, which concerns the exact direct path.)

4.7 Kernel scheduling and scoped pools

Four flop-count thresholds route the dense work, passed as a cheap `Copy KernelTuning` struct rather than global state: `scalar_gate` (≈ 4096 flops) below which an update is a scalar triple loop instead of a SIMD GEMM; `par_gemm` ($\approx 1 \times 10^6$) above which a `cmod` GEMM goes rayon-parallel; `par_cdiv` ($\approx 8 \times 10^6$), the top-of-tree node-parallelism lever, above which the panel-trailing/front GEMM goes parallel; and `use_gemm_schur`. The tree recursion runs in a *scoped* rayon pool of the requested width, so many concurrent solves coexist without oversubscribing the global pool, and the worker stack is sized to the (iteratively computed) assembly-tree depth, stack $=$ clamp(32 KB $\cdot$ depth, 16 MB, 8 GB), which is what lets deep banded chain-trees factor without a stack overflow.

5 A-priori resource model

The estimator is the foundation the tuner and the governor stand on: because peak memory is a *deterministic function of the symbolic structure*, it can be bounded before any numeric work, and that bound is what makes a learned tuner safe to ship as the default.

5.1 Per-path memory estimate

`estimate_memory::<T>()` returns a `MemoryEstimate` whose fields are all exact functions of the pattern and scalar size $v =$ `value_bytes`. The factor is `factor_nnz` $\cdot\,(v + 8)$ (the $+8$ is the CSC index). The two paths get *separate* transient bounds:

Left-looking. A refcount simulation of the panel-freeing schedule (Sec. 4) gives `panel_live_peak_bytes` — the tightest bound on live panel memory, the "floor" the solver actually operates at. A conservative whole-run bound assumes the parallel frontier holds all panels at once:

$$\text{scratch} = \tfrac{1}{4}\big(\text{panels\_all}+\text{factor}\big)+32\,\text{MB}, \qquad \text{transient} = \text{panels\_all}+\text{factor}+\text{input}+\text{scratch}, \quad (2)$$

where the $\frac{1}{4} + 32\,\text{MB}$ floor covers per-thread `cmod`/`cdiv` and emit buffers. It is a genuine upper bound: over the corpus the estimate/measured ratio is ≈1.5 in geomean and *never* under-predicts (Fig. 7), which is the property the memory guarantee needs. The panel-freeing floor is the tighter quantity used inside the tuner's veto.

Multifrontal. A level-parallel CB-stack model: at each assembly-tree level the peak holds all fronts of that level ($\sum \text{nrow}^2 v$) plus the contribution blocks of their children ($cb(s) = \text{cnrow}^2 v$). A $\times\frac{7}{5}$ (1.4×) overlap margin accounts for the work-stealing scheduler running deep leaves of one subtree alongside mid-level fronts of another, keeping the bound above the measured 24-thread peak.

The geometric work `factor_flops` $= \sum_s \text{nrow}_s^2\,\text{ncol}_s$ is a type-independent proxy that lets a single `f64` calibration serve all scalar types.

5.2 Runtime estimate, calibration, thread predictor

Wall-clock is factor_flops/(gflops · speedup), where the per-machine `Calibration` measures the single-thread geometric-flop rate and the parallel speedup once, by factoring a 24^3 7-point Laplacian, and caches the result keyed by a hardware fingerprint (FFTW-"wisdom" style [14]); `speedup_for(threads)` interpolates linearly to the calibrated peak and holds flat beyond it, matching the saturating scaling of sparse-direct work. The thread-count predictor (`recommend_threads`) maps three structural features — factor flops, largest front height, peak tree width — to a worker count: thin-and-narrow problems (front_nrow_max < 512 and tree_width_max < 128) are capped at 2 workers where more only regress; tiny problems (factor_flops $< 3 \times 10^8$) at 4; the rest use all cores. On the corpus this lands within ∼10% of the per-matrix optimal (geomean), against ∼50% for a fixed budget of 2.

5.3 Structural features

The auto-tuner consumes a 20-dimensional structural fingerprint (extended to the 32-dimensional network input by the knob encoding, Sec. 6): pattern statistics computed in $\mathcal{O}(\text{nnz})$ — n, nnz, mean/max degree, degree coefficient of variation (arrow detector), max and relative-mean bandwidth, diagonal-dominance and diagonal-presence fractions — and post-analysis symbolic statistics — supernode count, fill nnz and fill ratio, mean supernode width, max front height, tree depth, max/mean tree width, factor flops, arithmetic intensity (flops per stored entry, the BLAS-3-richness signal), and the fraction of flops in the largest / top-1% of fronts (top-of-tree heaviness). These are the quantities that decide which knob setting wins, so they are exactly what the model sees.

6 The learned auto-tuner

6.1 Performance model

A small multilayer perceptron predicts (log factor_ms, log peak_mb) from the standardized structural features concatenated with an encoding of a candidate knob vector (ordering one-hot over {auto, amd, metis}, `nemin`, relaxation width, `panel_nb`, `scalar_gate`, `par_cdiv`, method): a $32 \to 64 \to 32 \to 2$ network with ReLU hidden activations, trained offline (3054 measured (matrix, config) samples from the knob sweep) with scikit-learn [15] and exported as JSON weights.

Inference is pure Rust — a few dense mat-vecs — run at analyze time over a fixed candidate grid; the output is destandardized to milliseconds and megabytes. The chosen configuration minimizes a weighted Pareto objective

$$\text{score}(w) = w\,\log(\hat{t}) + (1-w)\,\log(\hat{m}), \tag{3}$$

with the default weight $w = 0.7$ leaning toward speed while still valuing memory; $w = 1$ and $w = 0$ give the pure speed and pure memory modes.

6.2 The guard stack: speculation with a safety net

The learned prediction is treated as *speculative*. Analogous to a CPU branch predictor [16] — where a misprediction costs only a pipeline flush, never correctness — the tuner's mistakes are made cheap by a deterministic guard stack, so the model can be an always-on default rather than an opt-in gamble. A candidate is only accepted if it clears, in order:

1. **Out-of-distribution fallback.** If factor_flops exceeds the training cap `flops_ood_cap` ($\approx 1.42 \times 10^{10}$, the largest flop count seen with enough configs in training), return the untuned default — the model does not extrapolate.
2. **Hard predicted-memory ceiling.** A candidate whose predicted log peak_mb exceeds the default's by more than `MEM_TOL_LN` $= \ln(1.02)$ is rejected.
3. **Multifrontal floor veto.** A multifrontal candidate is rejected outright when the ratio of its estimated transient to the left-looking floor exceeds `VETO_MF_MEM_RATIO` $= 1.0$ — the deterministic estimate, not the model, vetoes memory-pathological MF picks.
4. **Deterministic a-priori backstop.** The chosen configuration is re-analyzed and accepted only if its *exact* estimates satisfy fill $\leq 1.02\times$ default, flops $\leq 1.05\times$ default (a time proxy), and its realistic memory floor stays under the default's (multifrontal transient vs. the LL floor, or LL floor vs. floor); otherwise the default is used.
5. **Gain guards.** The surviving candidate must beat the default by at least `MIN_GAIN` $= 0.08$ in score, or `METHOD_FLIP_GAIN` $= 0.20$ if it changes the factorization method — so noise-level "wins" and risky method flips do not displace a working default.

The asymmetry is deliberate and reflects what is knowable: *memory* is a deterministic function of structure, so it can be *hard-guaranteed* never worse; *time* depends on BLAS-3 efficiency, which the model predicts only approximately, so it is optimized in the geomean with an unavoidable error tail but backed by the exact-flop proxy. Memory is the critical resource and is treated as such.

7 Deterministic resource governance and placement

Because the memory estimate is a hard a-priori *bound*, not a guess, the solver can be scheduled to the edge of a resource budget without the safety margin every other solver must leave. The single-solve governor `tuning::plan` already turns a `MemoryEstimate` + a `Budget` (RAM ceiling, thread budget) + the hardware calibration into a `FactorPlan` with a predicted peak, a predicted wall-clock, and a `fits` decision — purely as a function of its inputs, hence reproducible.

The natural extension, and the payoff for batched workloads, is a *deterministic batch scheduler*: a 2-D bin-packing of concurrent solves into a (RAM, cores) budget. Each solve consumes $(\text{mem}_i, \text{threads}_i)$; the memory axis is bounded exactly and the thread axis is the second free variable (the thread predictor gives each solve a scaling-aware width, so non-scaling solves run many-abreast at low thread counts while a big BLAS-3-rich solve claims the cores). Since the peaks are guaranteed, the packing can fill RAM to the edge without the OOM risk that forces others to under-provision — an OOM kill mid-batch on rented cloud hardware is wasted money — and

since the wall-clocks are predicted, the schedule is makespan-aware and the whole batch's cost is computable *before* the machine is provisioned. This is the "full, deterministic utilization" use case: rent a large box for a day, pack a batch of (e.g. curl–curl) simulations to saturate both RAM and cores, know the runtime and cost up front. Three honest caveats remain — memory-bandwidth contention limits how far independent factorizations actually scale abreast (the calibration models per-solve speedup, not contention), the estimator's conservatism margin costs packing density (a per-class calibrated estimate tightens it), and NUMA pinning matters on large sockets — and are the subject of the batch-scheduler work item.

8 Evaluation

All figures are produced by the `bench_suite` corpus engine and `superlu_corpus.py` over the SuiteSparse collection [24] (1k–100k DOFs; SPD, indefinite, unsymmetric, complex) on a 12-core / 24-thread machine, in paper mode (black axes). RSLAB, faer [22], MKL PARDISO [21] and SuperLU [23] were measured in *one run* so the cross-solver numbers are drift-free.

8.1 Scaling against faer, PARDISO, SuperLU

Figures 1 and 2 plot factor time and peak memory against nnz with a least-squares power-law fit $\sim \text{nnz}^{\alpha}$ per solver (residual > 0.1 excluded from the fit). The RSLAB curve is the *auto-tuned default* — the product as shipped, picking left-looking or multifrontal per matrix under the guards — not the raw kernels.

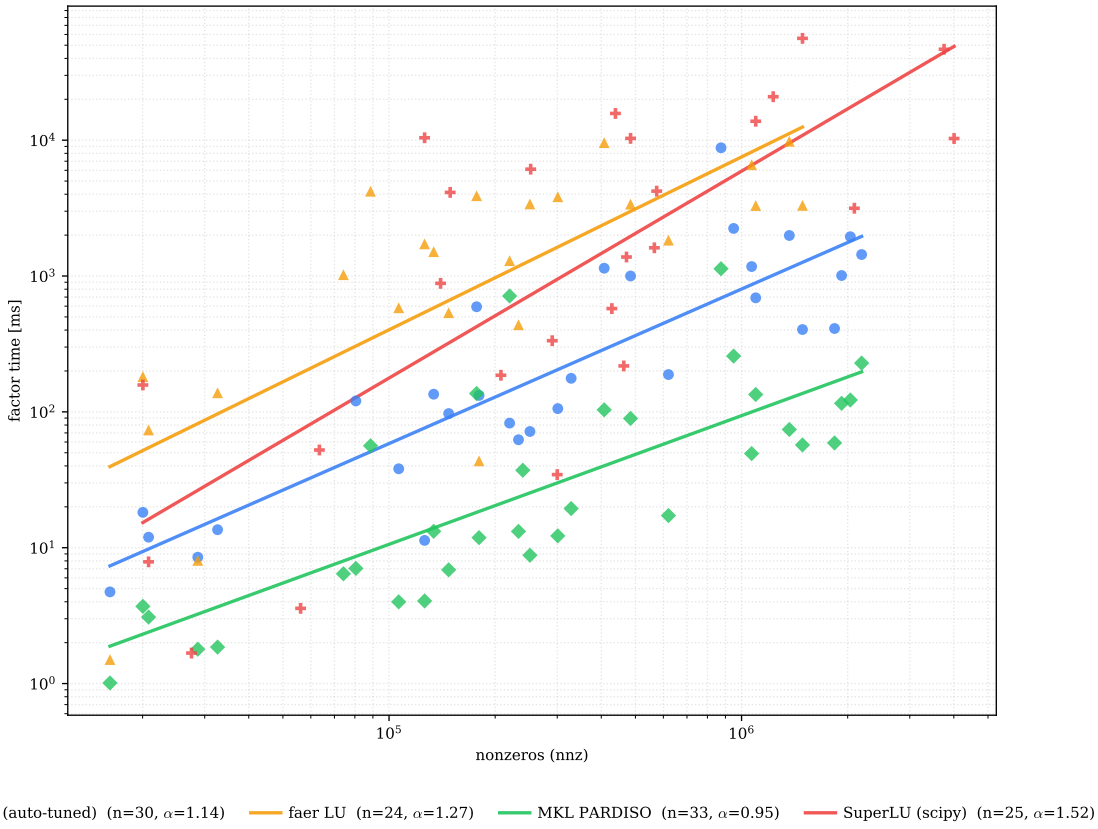

Figure 1: Corpus scaling of factor time vs. problem size, with power-law fits. RSLAB (auto-tuned, black) sits between PARDISO and faer.

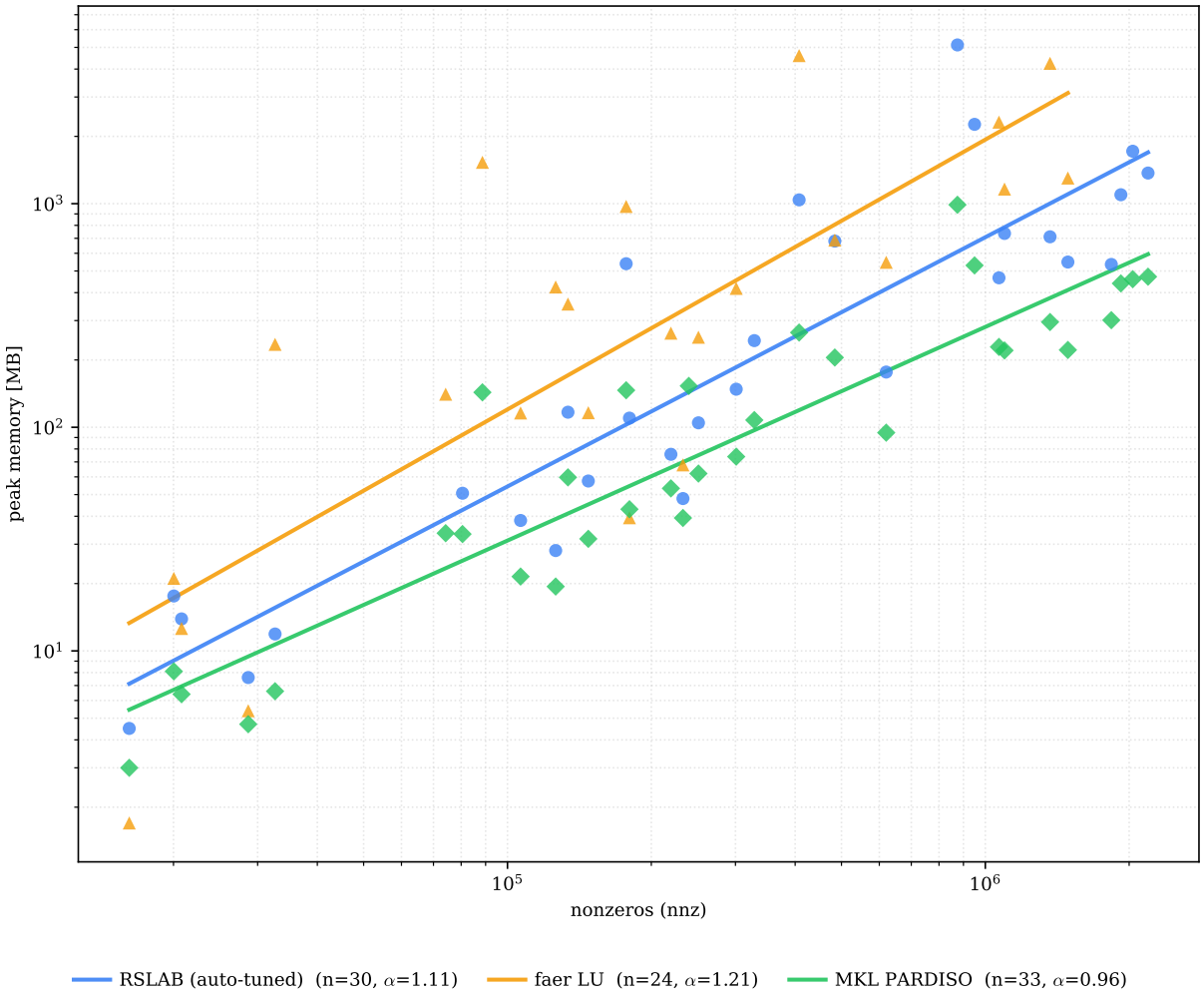

Figure 2: Corpus scaling of peak factor memory vs. problem size, with power-law fits.

8.2 Accuracy

Figure 3 shows the relative residual $\|Ax-b\|/\|b\|$ per solver. Where RSLAB factors it is accurate — 24 of 31 matrices below 1×10^{-8}, matching PARDISO and ahead of faer, which returns a degraded solution on several. The exact $\mathbf{LDL}^\top$ (pivoting bounded per supernode) declines a handful of indefinite saddle-point matrices rather than returning a wrong answer.

8.3 The auto-tuner end to end

Figure 4 measures the tuned configuration (picked from structural features alone, under the full guard stack) against the untuned default across the corpus. The balanced tuner ($w = 0.7$) is 1.38× faster at 0.81× the memory, at *zero* of 89 memory regressions; the gain is larger on small matrices (1.57×) than large (1.22×), and the Pareto weight slides the operating point between speed and memory.

8.4 Thread scaling and memory-estimate validation

Figure 6 shows per-solver thread scaling (mean over the corpus with a min–max band): sparse-direct factorization is largely memory-bandwidth bound, so every solver saturates well below linear, and the wide band — some matrices *regress* past a few threads — is why RSLAB predicts the worker count per matrix. Figure 7 validates the a-priori estimator — the conservative bound and

	scaling order α		head-to-head vs. RSLAB (geomean)	
solver	factor time	peak memory	factor time	peak memory
MKL PARDISO	**0.95**	**0.96**	7.1× slower	2.3× more
RSLAB (auto-tuned)	1.14	1.11	—	—
faer LU	1.27	1.21	7.0× faster	2.3× less
SuperLU (SciPy)†	1.52	—	8.9× faster	—

Table 2: Empirical scaling orders and head-to-head geomean ratios (matrices both solvers factor to < 0.1 residual). RSLAB beats faer on both axes and trails only PARDISO's decades-tuned sublinear order. †SuperLU is included only as an open-source reference and only on the time axis: its peak memory is sampled process RSS from a SciPy child (`splu` exposes no peak), not comparable to the in-process peaks and unreliable, so it is omitted from the memory comparison; and it runs single-threaded with a 60 s per-matrix cap, so its 8 hardest corpus matrices are absent (survivor bias flatters its time), and it scatters over three orders of magnitude.

the panel-freeing floor against the measured peak of both RSLAB paths (it never under-predicts) — and Figure 8 shows where each path spends its wall-clock.

9 Related work

Learned and automatic solver tuning has prior art, but along different axes. *GPTune* [17] (LBNL) tunes HPC libraries including SuperLU_DIST by Bayesian optimization with multitask learning across problem sizes — more sophisticated in its surrogate and uncertainty handling, but requiring many online trial runs per problem; RSLAB's feed-forward model runs once at analyze time with negligible overhead. *OSKI* [18] autotunes at the sparse-*kernel* level (register/cache blocking for SpMV), not solver configuration. *Lighthouse* [19] and the work of Bhowmick, Raghavan et al. [20] apply machine learning to *select* a solver or preconditioner from a portfolio (classification/recommendation), not to predict and tune a direct factorization's configuration. Production direct solvers — MKL PARDISO [21], MUMPS, cuDSS — ship *hand-written* heuristic "auto" modes (e.g. automatic reordering choice), not learned models. What is uncommon in combination, and to our knowledge not shipped elsewhere, is a learned performance *regressor* used as the always-on default of a sparse direct solver, made safe by a deterministic memory guarantee and an OOD fallback, in a dependency-free, type-agnostic, pure-Rust implementation. The closest conceptual relative is not in numerical software at all but in hardware: the perceptron/neural branch predictor (Jímenez–Lin [16], and the neural predictors shipped in modern CPUs), which likewise pairs a learned speculation with a mechanism that makes misspeculation free of cost.

10 Limitations and future work

The gains are aggregate: 1.38× geomean, part of it within the $\sim \pm 20\%$ per-matrix single-shot measurement noise, so the geomean — not any single matrix — is the signal. The model is trained on one machine and one corpus and does not transfer across hardware without retraining; this is the same per-microarchitecture calibration a branch predictor needs, and motivates a *meta-tuner* that runs the sweep + training + guard-threshold auto-calibration on a user's own matrix set to emit a problem-class profile. Time is optimized but not hard-guaranteed (only memory is). Planned tunable axes that keep the exact direct path — RCM/band ordering [25], exposed threshold-pivot tolerance, tunable equilibration strategy [26], static pivot-order reuse for fixed-pattern sequences, parallel/blocked triangular solves, 2-D front parallelism — each add a candidate-grid dimension the tuner learns, with the standing constraint that every new axis ships an a-priori memory estimator so the deterministic backstop keeps holding.

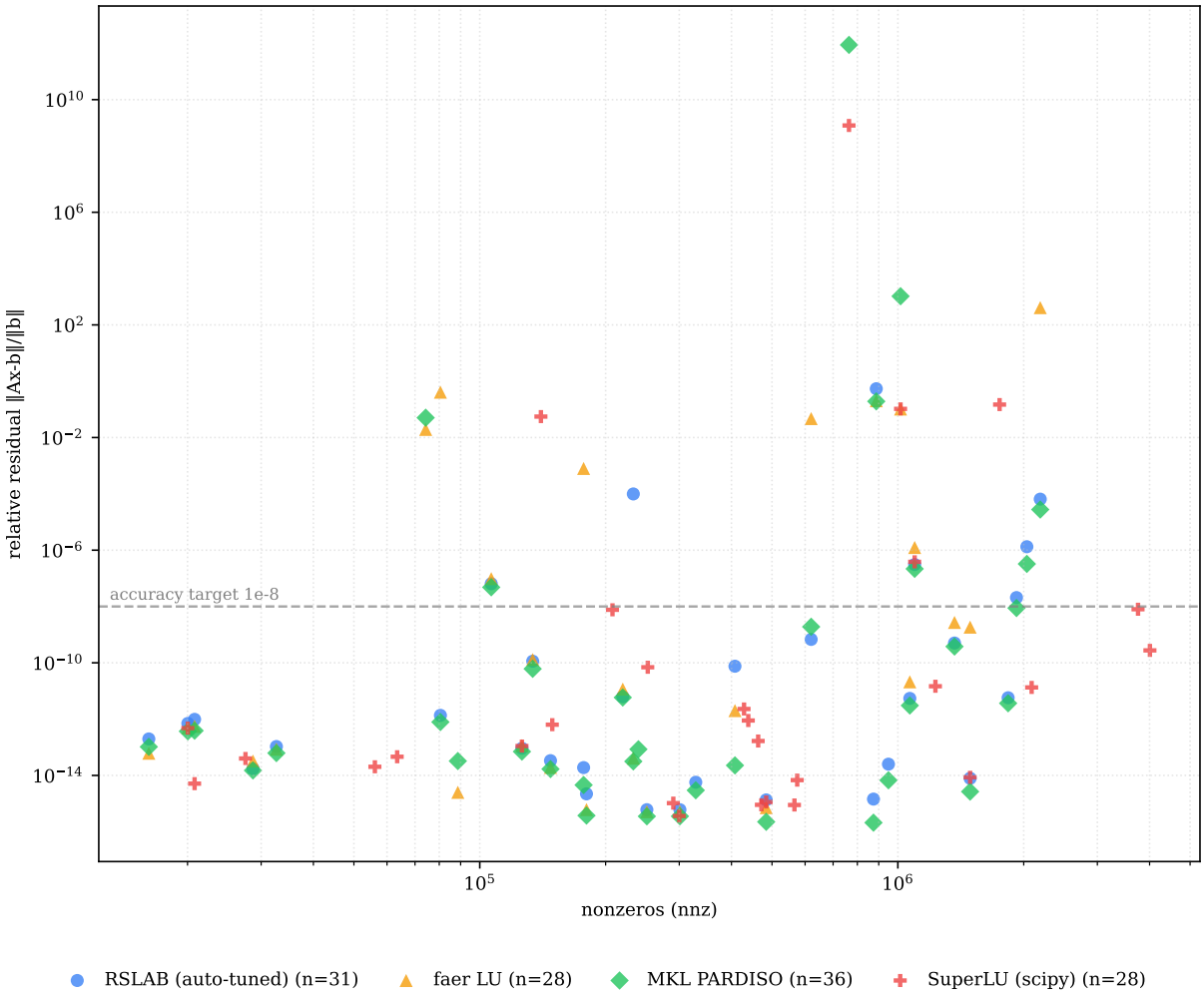

Figure 3: Relative residual across the corpus, per solver.

11 Conclusion

RSLAB is a pure-Rust, type-agnostic sparse direct solver that reaches a single-digit factor of MKL PARDISO and clearly outperforms the open-source alternatives, with a learned auto-tuner as its default configuration path. The contribution is less the raw speedup than the architecture: a per-path a-priori memory estimator accurate enough to serve as a hard bound, a learned performance model that speculates on the best knob vector, and a deterministic guard stack that makes that speculation provably free of memory cost — branch prediction for a sparse solver. That same estimator turns batched solving into a deterministic packing problem, where a rented machine's RAM and cores can be filled to the edge with the runtime and cost known in advance.

References

[1] P. R. Amestoy, T. A. Davis, I. S. Duff. An approximate minimum degree ordering algorithm. *SIAM J. Matrix Anal. Appl.* 17(4):886–905, 1996; and Algorithm 837: AMD. *ACM Trans. Math. Softw.* 30(3):381–388, 2004.

[2] P. R. Amestoy. Recent progress in parallel multifrontal solvers and the approximate minimum fill (HAMF) ordering. Technical report / CERFACS, 1999.

[3] E. Rothberg, S. C. Eisenstat. Node selection strategies for bottom-up sparse matrix ordering. *SIAM J. Matrix Anal. Appl.* 19(3):682–695, 1998.

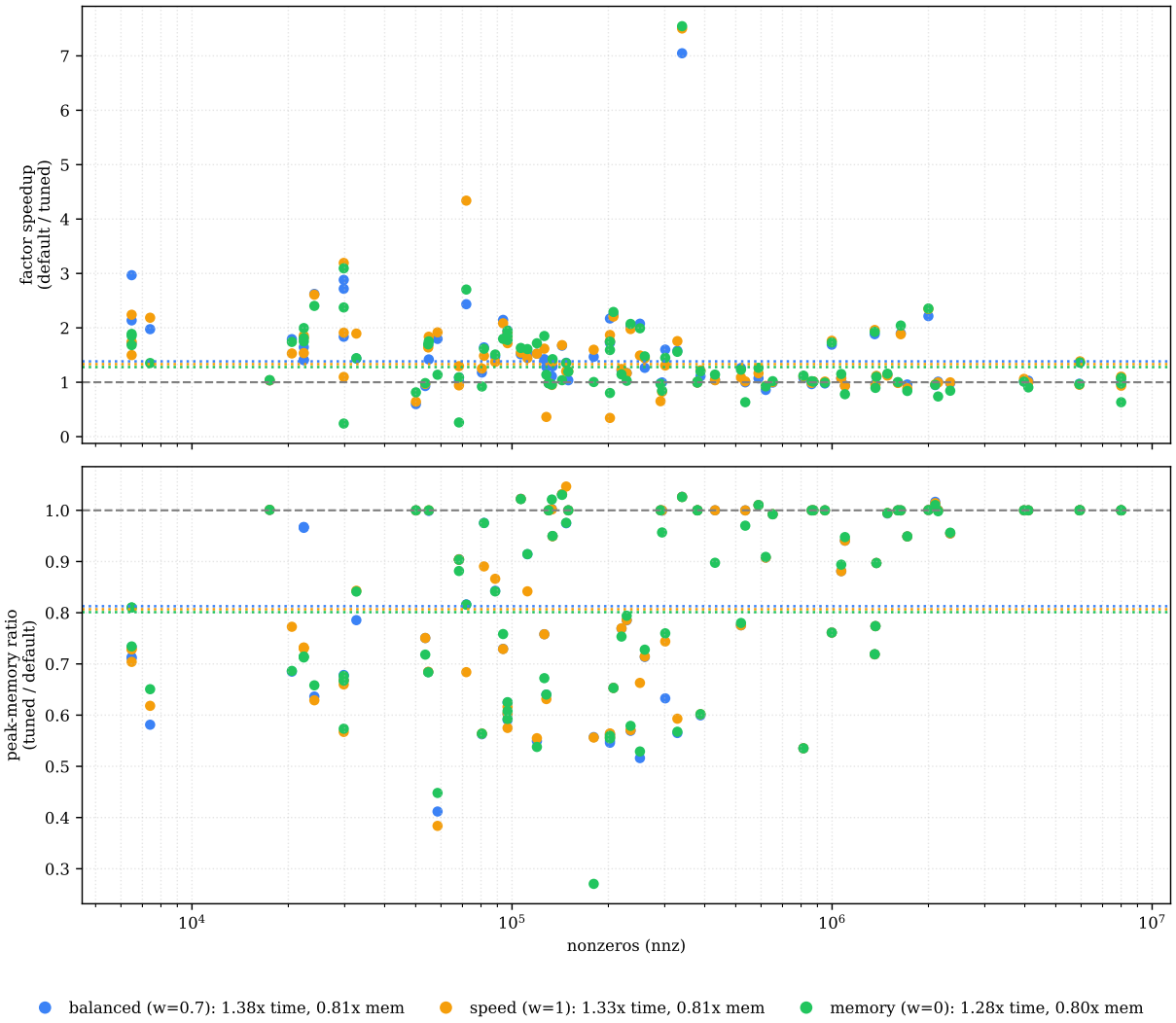

Figure 4: Factor speedup and peak-memory ratio of the tuned vs. untuned solver by problem size, three Pareto weights (dotted = geomean). Memory never regresses.

[4] A. George. Nested dissection of a regular finite element mesh. *SIAM J. Numer. Anal.* 10(2):345–363, 1973.

[5] G. Karypis, V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM J. Sci. Comput.* 20(1):359–392, 1998.

[6] C. M. Fiduccia, R. M. Mattheyses. A linear-time heuristic for improving network partitions. *19th Design Automation Conference*, 175–181, 1982.

[7] J. D. Hogg, E. Ovtchinnikov, J. A. Scott. A sparse symmetric indefinite direct solver for GPU architectures (SSIDS). *ACM Trans. Math. Softw.* 42(1):1–25, 2016.

[8] C. Ashcraft, R. Grimes. The influence of relaxed supernode partitions on the multifrontal method. *ACM Trans. Math. Softw.* 15(4):291–309, 1989.

[9] J. R. Bunch, L. Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Math. Comp.* 31(137):163–179, 1977.

[10] I. S. Duff, J. K. Reid. The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Trans. Math. Softw.* 9(3):302–325, 1983.

[11] J. W. H. Liu. The multifrontal method for sparse matrix solution: theory and practice. *SIAM Review* 34(1):82–109, 1992.

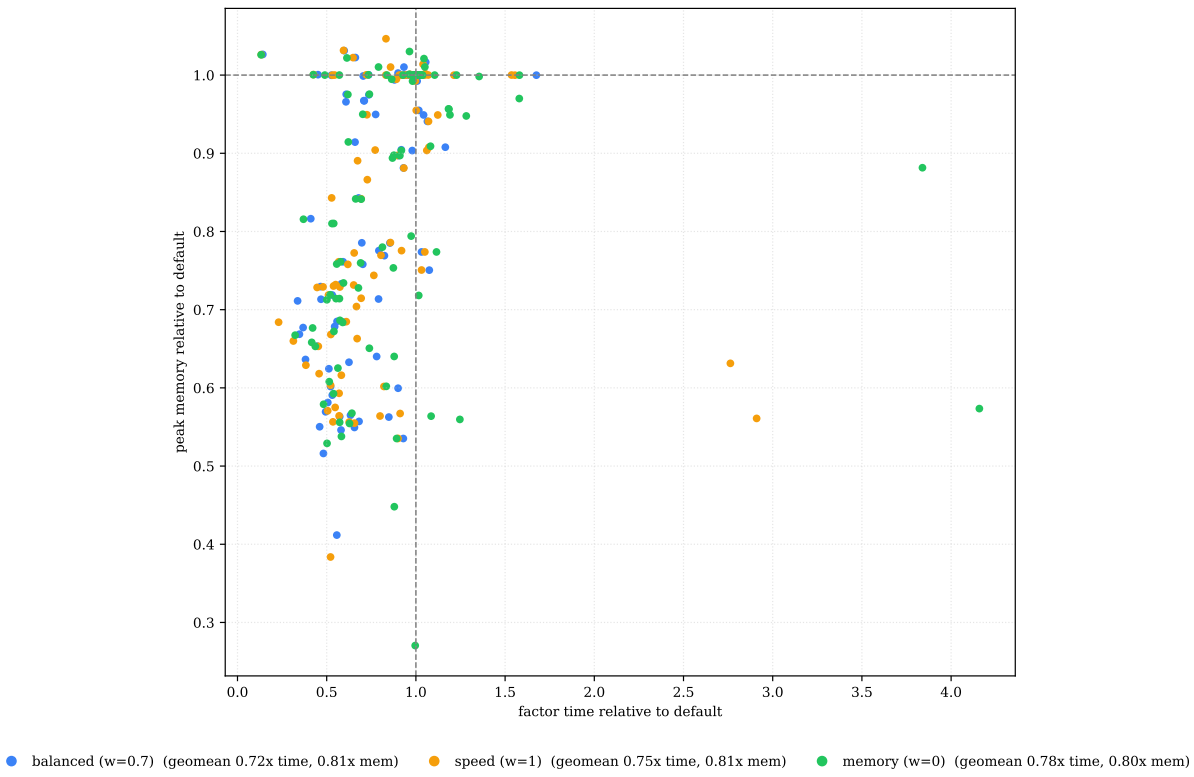

Figure 5: The auto-tuner's three Pareto modes as a time–memory cloud relative to the untuned default; the weight w slides the operating point.

[12] J. W. H. Liu. On the storage requirement in the out-of-core multifrontal method for sparse factorization. *ACM Trans. Math. Softw.* 12(3):249–264, 1986.

[13] T. A. Davis. Algorithm 832: UMFPACK, an unsymmetric-pattern multifrontal method. *ACM Trans. Math. Softw.* 30(2):196–199, 2004.

[14] M. Frigo, S. G. Johnson. The design and implementation of FFTW3. *Proc. IEEE* 93(2):216–231, 2005.

[15] F. Pedregosa et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12:2825–2830, 2011.

[16] D. A. Jiménez, C. Lin. Dynamic branch prediction with perceptrons. *7th Int. Symp. High-Performance Computer Architecture (HPCA)*, 197–206, 2001.

[17] Y. Liu, W. M. Sid-Lakhdar, O. Marques, X. Zhu, C. Meng, J. W. Demmel, X. S. Li. GPTune: multitask learning for autotuning exascale applications. *PPoPP*, 234–246, 2021.

[18] R. Vuduc, J. W. Demmel, K. A. Yelick. OSKI: a library of automatically tuned sparse matrix kernels. *J. Phys.: Conf. Ser.* 16:521–530, 2005.

[19] E. R. Jessup, P. Motter, B. Norris, K. Sood. Performance-based numerical solver selection in the Lighthouse framework. *SIAM J. Sci. Comput.* 38(5):S750–S771, 2016.

[20] S. Bhowmick, V. Eijkhout, Y. Freund, E. Fuentes, D. Keyes. Application of machine learning in selecting sparse linear solvers. *Int. J. High Perf. Comput. Appl.*, 2006.

[21] O. Schenk, K. Gärtner. Solving unsymmetric sparse systems of linear equations with PARDISO. *Future Gener. Comput. Syst.* 20(3):475–487, 2004.

[22] S. Quinones. faer-rs: a linear algebra library for the Rust programming language. https://github.com/sarah-quinones/faer-rs, 2024.

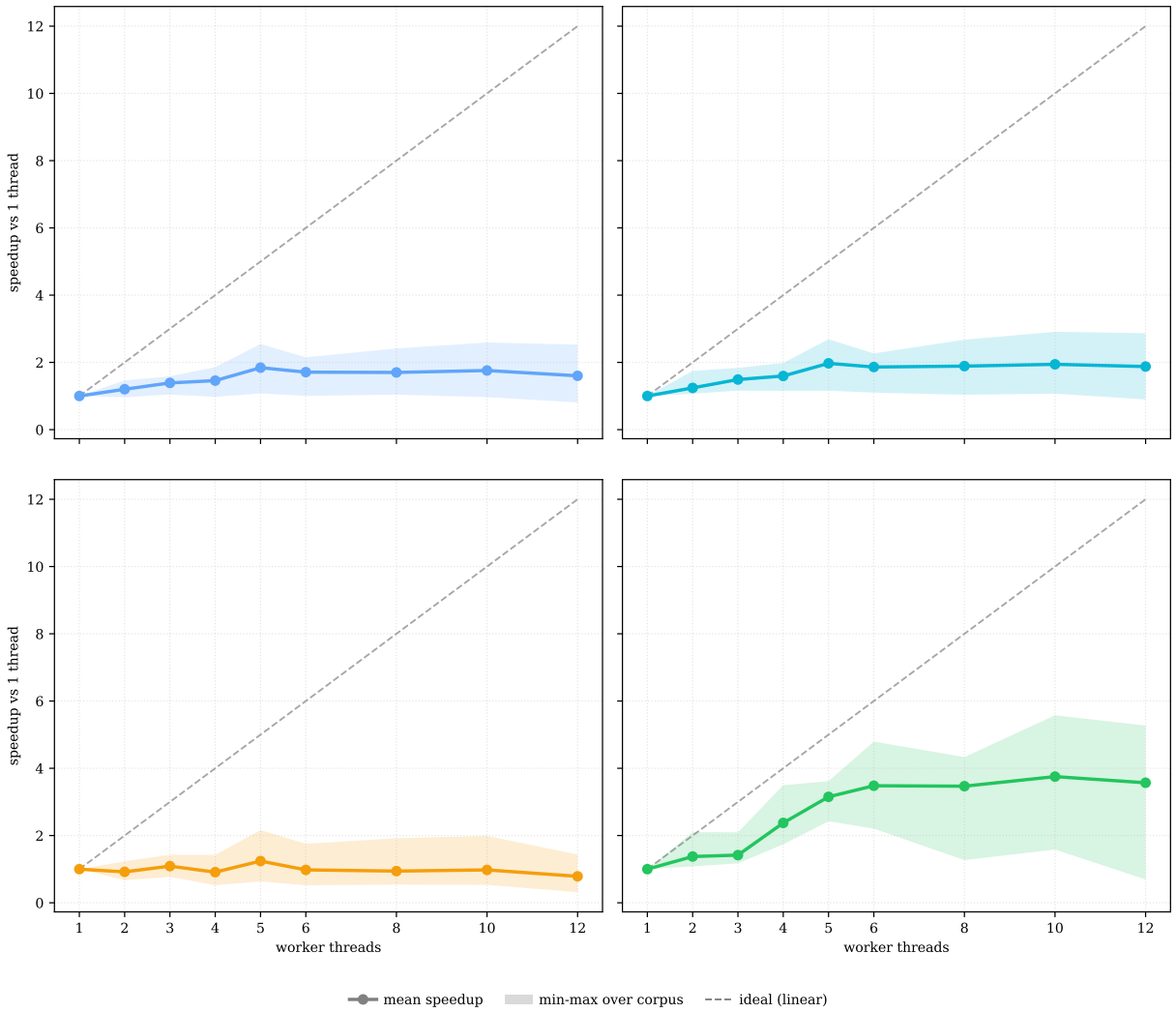

Figure 6: Thread scaling per solver, 1–12 workers, mean ± min–max over the corpus.

[23] X. S. Li. An overview of SuperLU: algorithms, implementation, and user interface. *ACM Trans. Math. Softw.* 31(3):302–325, 2005.

[24] T. A. Davis, Y. Hu. The University of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38(1):1–25, 2011.

[25] E. Cuthill, J. McKee. Reducing the bandwidth of sparse symmetric matrices. *Proc. 24th ACM National Conference*, 157–172, 1969.

[26] D. Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical Report RAL-TR-2001-034, Rutherford Appleton Laboratory, 2001.

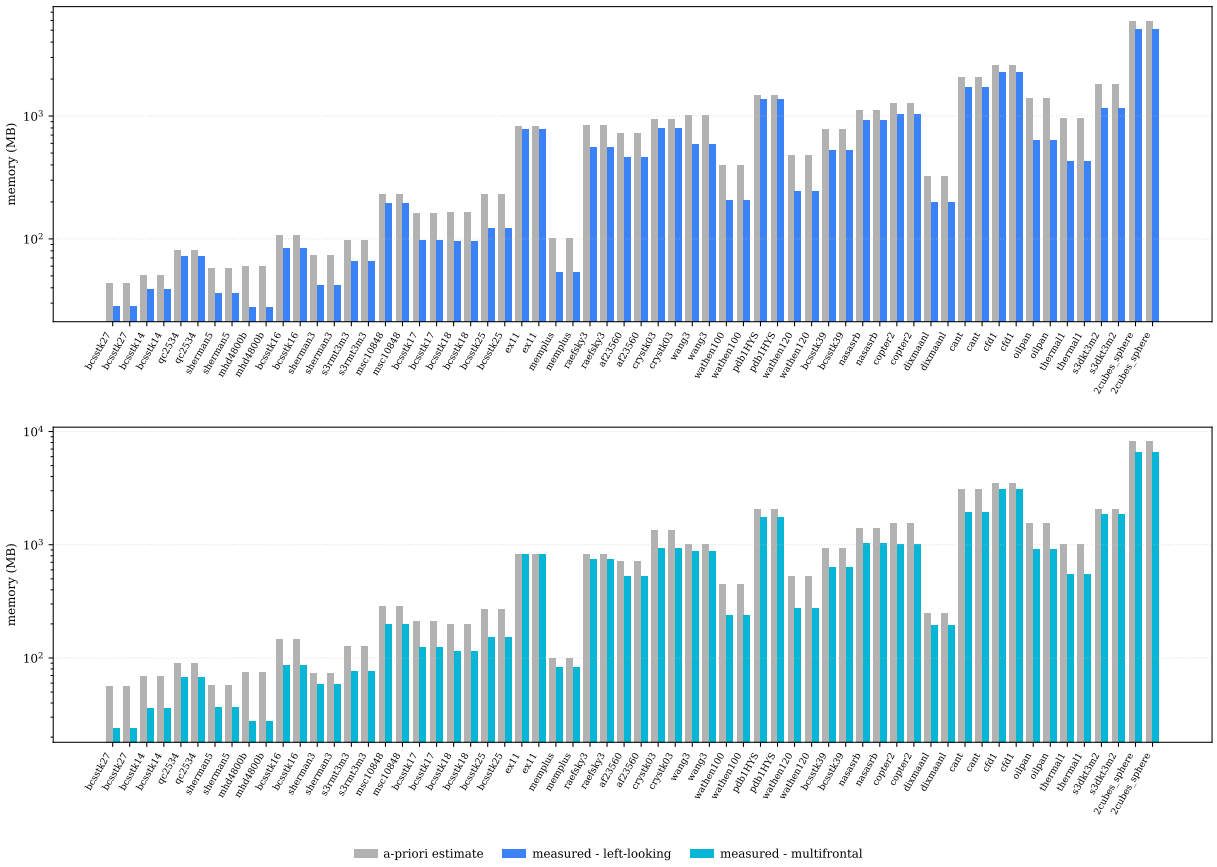

Figure 7: A-priori memory estimate (conservative bound + panel-freeing floor) vs. the measured peak, per RSLAB path (left-looking in black, multifrontal in gray). The estimate never under-predicts.

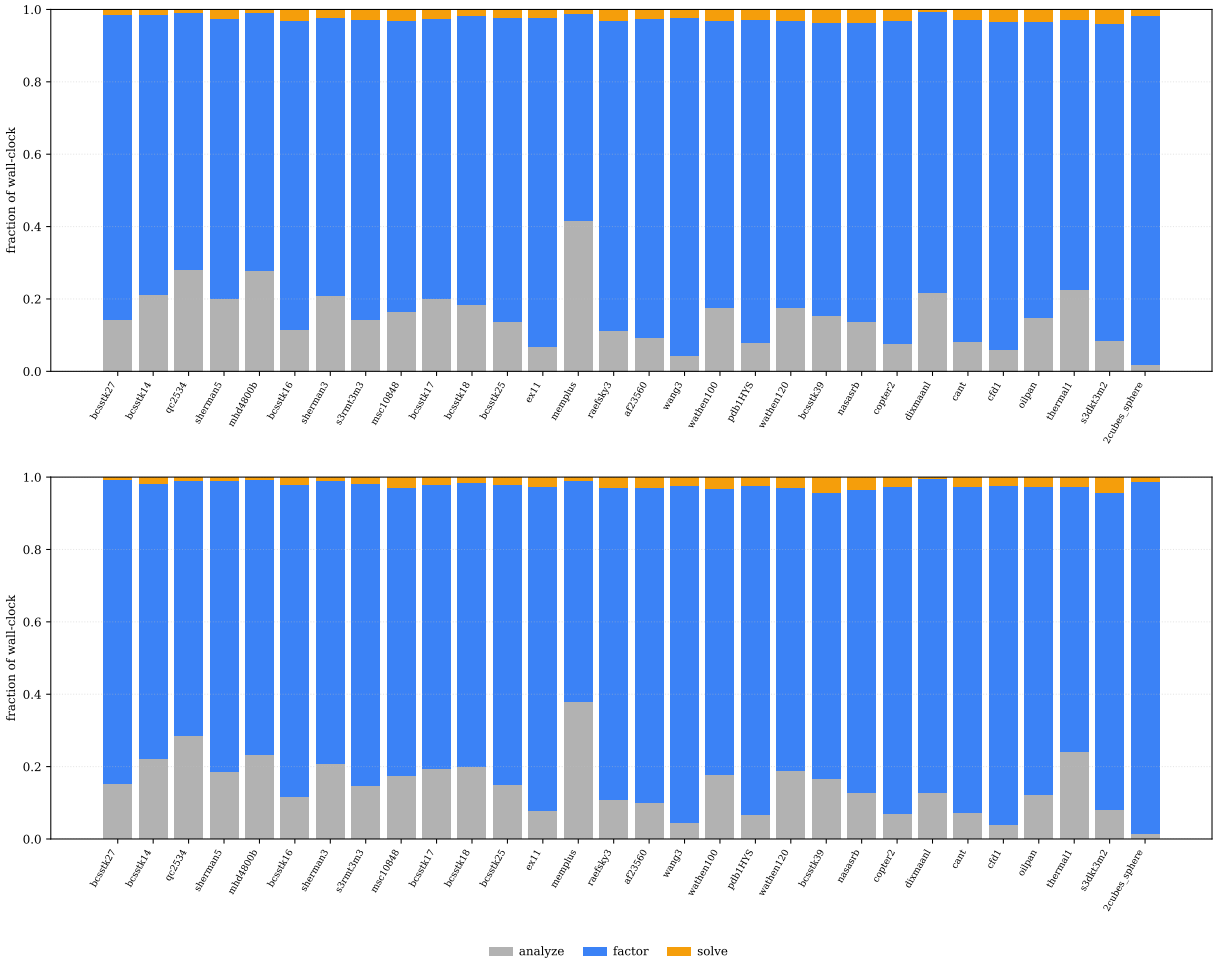

Figure 8: Analyze / factor / solve wall-clock split per matrix, per RSLAB path.